

微信个人情报库

wechat-intelligence-hub 小白指南

把本地微信聊天记录变成「可检索、可核查、可行动」的个人情报 —— 纯本地、只读、不 Hook 的微信开源工具实测手记

开源项目类

Python 3 引擎

SQLCipher / WCDB

AGPL-3.0

2.6k★

Windows 实测通过

来源：X 推文推荐（"遇到这种项目别犹豫，第一件事先 fork"）。仓库：github.com/Rion-Wu-tech/wechat-intelligence-hub。本指南按"工科毕业生第一次接触"的视角，从 13 个疑问一路讲到动手实验，**所有命令均在本机真实执行过**。

疑问 01

这是什么？一句话能说清吗？

一句话：**一个跑在你自己电脑上的"微信情报官"** —— 它读取你本机的微信聊天记录（只读！），自动整理出：今天哪些消息需要回复、你对谁许了什么承诺还没兑现、哪些聊天里藏着商单 / 合作 / 复联线索，最后生成一份 Markdown + 交互 HTML 的日报。

它不是微信机器人：不会替你发消息、不 Hook 微信进程、不需要把数据上传任何服务器。**数据不出电脑**是它的核心卖点，也是它敢开源的底气。



图 1：聊天记录 → 情报引擎 → 可行动的清单与报告

疑问 02

为什么需要它？没有它之前人们怎么做？

想象一个自由职业者 / 小团队老板的一天：

头疼场景	没有它的做法	有它之后
20 个群开了免打扰	一条条翻，翻完一小时	日报直接列出"群里出现的商单线索 + 证据原句"
上周答应客户"周五给初稿"	靠脑子记，忘了就丢单失信	承诺跟踪：谁、什么事、什么时候到期
"上次那个报价是多少来着？"	翻半小时聊天记录	按联系人 / 关键词秒级本地检索
半年没联系的老客户突然出现	完全没察觉这是商机	复联线索：识别"久未联系 + 出现合作意向"

它解决的不是"聊天太多"，而是"信息躺在聊天记录里睡大觉"。聊天记录是座矿山，它是一台本地选矿机。

疑问 03

微信聊天数据到底存在哪？为什么说是"堡垒"？

微信桌面版把消息存在**本机的加密数据库**里。要读它，得穿过三道门：

门	技术	一句话解释
■ 文件整体加密	SQLCipher	整个 SQLite 数据库文件被加密，没钥匙打开就是一堆乱码
■ 消息体压缩	WCDB + Zstandard	腾讯的 WCDB 库把消息内容 zstd 压缩后存库，解出还要再解压
■ 密钥授权	schema-2 salt-key	密钥不是"偷"来的，是你本人授权导出、由你自己提供的访问材料

所以"安装就能读所有人的微信"是**不可能的**——README 反复强调：**完整读取需要本人授权的本地数据库和访问材料，且只覆盖已同步到本机的数据**。这把"能不能读"从技术问题变成了授权问题，也是它与传统"微信多开 / Hook 工具"的本质分界。



图 2：读一条微信消息要连穿三道门

疑问 04

"Clean-room 只读 Reader"是什么？不 Hook 意味着什么？

Clean-room (洁净室) 设计: Reader 核心是一个独立实现的只读解析器，官方承诺 "不获取密钥、不重签名、不注入、不 Hook 微信"。

对比一下就明白分量了：

做法	原理	风险
本项目 Reader	解析静态数据库文件的只读代码	微信无感知，不违反进程边界
Hook / 注入	把代码塞进微信进程内存里拦截消息	微信可检测 → 封号风险；需要重签名
协议破解机器人	模拟微信客户端协议收发消息	明确违反用户协议，封号重灾区

macOS 上另有一个**实验性接入助手**（可能重启微信/重签名副本，需用户单独确认授权）；**Windows 的首次取钥尚未合入主仓库**——本机实测 `home` / `db-status` 在没有授权数据库时会被明确拒绝（见实验第 5 步）。

疑问 05

仓库说"一个产品，两个 Skill，四层架构"，具体长什么样？



为什么要拆四层？**依赖方向单一、可独立测试**：Reader 层不依赖引擎，引擎可以用**全虚构样例数据**跑通全部测试（本机已验证，Windows 原生 CI 同样只测虚构材料）。用户视角仍是一次安装。

疑问 06

引擎怎么从聊天里“识别商单信号”？是 AI 吗？

不是大模型，是确定性规则引擎——同一份输入永远得到同样的输出，这正是它被单元测试锁定的原因（与知识库里的确定性脚本是同一思想）。每条消息被规则打上三个标签：

标签	取值示例（实测 signals.csv 里出现的）
category 类别	合作意向 / 报价 / 结算 / 排期 / 创作 / brief / 待回复
stage 阶段	新线索 → 待报价 → 待确认报价/排期 → 待创作 → 已发布待结算
priority 优先级	1~5，"待回复"给 5（最急），"新线索"给 2

实测一条消息能产生**多条**信号（“预算 650 USD，想这周五发布”同时命中 报价+排期+创作 三条）。最后按聊天对象聚合，输出“今天优先处理”清单，每条都**附证据原句**——这正是它和 [[AI 味模式库与信号非证据]] 里“信号不是证据”的呼应：**每条判断都必须能点开原句核查**。

疑问 07

一份日报是怎么从无到有生成的？



图 4：日报流水线（数字为官方虚构样例的实测值）

本机实测官方样例（`run_demo.sh`）：读取 **8 条**消息 → 识别 **21 条**信号 → 聚合 **5 个**聊天对象 → 输出 `signals.csv` / `deals.csv` / `daily_digest.md` 三件套。

疑问 08

HTML 报告为什么要 pandoc + nh3 两个工具？

日报默认是 Markdown。要变成能发给别人的独立 HTML，需要两步：

① **pandoc**：万能文档“翻译机”，把 Markdown 转成 HTML5 片段；② **nh3**：HTML 消毒器（基于 Mozilla 的 ammonia），把转出来的 HTML 按白名单**洗掉脚本和危险标签**——防止聊天记录里有人发过恶意 HTML/脚本，一转报告就在你浏览器里执行（XSS）。

缺 pandoc 时它明确报错、不降级：README 写明“HTML 生成会明确报错，不降级为未净化页面”——宁可没有，不给脏的。本机通过 `py pandoc-binary` 拿到 pandoc 3.9 后实测渲染成功（实验第 4 步）。

疑问 09

Windows 上能直接用吗？能读到我的真微信吗？

分两层回答（都是 README + 实测的结论）：

能力	Windows 状态
跑虚构样例 / 读聊天导出文件 / 生成日报	✅ 开箱即用（本次全链路实测通过）
SQLCipher 解密、WCDB 解压（读取已授权数据库）	● CI 已验证，需你提供授权 key
首次自动获取密钥（取 key）	❌ 尚未合入主仓库，未完成新电脑真机验收

一句话：引擎和报告层 Windows 随便玩；读真库要自备授权材料；指望它自动破解微信，不存在的。这也解释了推文为什么喊“先 fork”——微信相关工具随时可能 404，先把代码留下。

疑问 10

Codex Skill 在这里是什么角色？

仓库带两个 "Skill" 目录（`skills/wechat-cli`、`skills/wechat-intelligence-hub`），本质是**给编程智能体看的操作规程**：什么时候该跑 `scan`、什么时候该跑 `access-plan`、哪些操作必须先问用户确认（如取 key）。所以 README 推荐的使用姿势是**把一整段话丢给 Codex**，让它自己安装、自检、报缺口，而不是人逐条敲命令。这和知识库里的 [[AgentSkills技能包]] 是同一范式：Skill 不负责执行，负责**教会 Agent 何时做什么、边界在哪**。

疑问 11

AGPL-3.0 许可证意味着什么？

AGPL 是 GPL 的“联网加强版”：只要你通过网络提供服务给别人用，就必须把完整源码也公开。对个人本地自用完全没影响；但如果哪家公司想把它集成进闭源商业产品，得找作者买商业授权（仓库里确实挂了 `COMMERCIAL-LICENSE.md`）。作者用 AGPL + 商业授权双轨，是典型的“开源留名、商用收费”路线。

疑问 12

实测：规则引擎的“视力”到底怎么样？

本指南最有价值的一节——用自建样例（实验第 3 步）测试它的精度边界：

消息	引擎的判断	评价
"尾款 12000 国庆后安排"	结算 · 已发布待结算 · 优先级 5	✓ 精准
"首款 40% 下周打款"	结算 · 优先级 5	✓ 精准
好久没联系的老同学来聊合作	合作意向 · 新线索	✓ 复联线索识别到位
妈妈："记得提前买票" / 我："回的，明天买"	排期 · 待确认报价/排期	✗ 误报：买票被当成商务排期
读书会："周五晚讨论，自愿报名"	排期 · 待确认报价/排期	✗ 误报
我："改好了，晚上发你"（对阿 may 的承诺）	（无信号）	⚠ 漏报：承诺句没被标记
猎聘 JD、快递通知	（正确忽略）	✓ 抗干扰

结论：确定性规则的代价是“宁可多报，不可漏报”（误报 3、漏报 1、抗干扰正确）。 它的定位是“筛子”不是“裁判”——把人从翻记录里解放出来，最终判断仍由人看证据原句完成。这与 [[事实与判断分离]] 的分层思想一致：引擎产事实（带证据的信号），人做判断。

疑问 13

它和知识库已学过的概念怎么连起来？

① 引擎是 [[确定性脚本]] 的"情报版"实例：同输入同输出，测试可锁定；② 报告管线是 [[构建流水线与 Pass]] 的迷你形态：读入→打标→聚合→写报→渲染，各 Pass 独立；③ "每条信号附证据原句"正是 [[AI味模式库与信号非证据]] 的铁律——信号不是证据，证据是可核查的原句；④ 给 Codex 的 Skill 是 [[AgentSkills 技能包]] 的实战样本；⑤ 引擎产事实、人做判断，是 [[事实与判断分离]] 的小型实践。

这个项目给我什么

① 聊天导出 → 商单日报

给一个按 [时间] 发送人: 内容 格式的导出文件，秒出"今天优先处理"清单 + 证据原句（本次实验实测）。

② 承诺与待回复跟踪

扫出"对方问了你但还没回"你答应了还没做"的事项，按优先级排序。

③ 商机雷达 + 复联线索

识别报价/预算/合作意向金额（实测能抽出"650 USD""3000-5000 RMB"）；找出久未联系但出现合作苗头的关系。

④ 本地情报库检索

把重点联系人/群聊索引进本地 SQLite 情报库，按人、按关键词、按日期秒查历史。

⑤ 净化 HTML 报告

Markdown 日报一键转独立 HTML（pandoc 转换 + nh3 白名单消毒），可直接发给协作者。

⑥ 给 Agent 的操作规程

两个 Skill 目录可直接教 Codex/Claude 何时用哪条命令、哪些动作必须先经人确认——可以搬来做自己项目的 Agent 操作手册模板。

动手实验：不碰真微信，跑通"扫描 → 日报 → HTML"全链路

共 5 步，全部在本机（Windows 11 + Python 3.14）真实执行通过。实验材料在 `F:\reminder\wechat-intelligence-hub\exercise\`。

第 1 步 · 克隆仓库并跑官方虚构 Demo

目的：零风险验证引擎主链路（官方自带全虚构样例，不碰任何真实数据）。

```
git clone https://github.com/Rion-Wu-tech/wechat-intelligence-hub repo
cd repo/projects/wechat-intelligence-hub
# Git Bash 下 PYTHON_BIN 指向 python (Windows 没有 python3)
PYTHON_BIN=python bash scripts/run_demo.sh
```

✓ 预期结果：输出"读取消息：8 条 / 识别信号：21 条 / 聚合对象：5 个"及输出目录；fake-vault 部分输出"3 条消息 → 9 条信号"。本机实测完全一致。

第 2 步 · 补齐 HTML 渲染依赖（可选但推荐）

目的：让第 4 步的 HTML 报告能跑。nh3 直接 pip 装；pandoc 用 py pandoc-binary 自带的 3.9。

```
pip install nh3 zstandard py pandoc-binary
python -c "import py pandoc; print(py pandoc.get_pandoc_path())"
# 把自带 pandoc.exe 放进 PATH (Git Bash)
cp "$APPDATA/Python/Python314/site-packages/py pandoc/files/pandoc.exe" ~/.local/bin/
export PATH="$HOME/.local/bin:$PATH" && pandoc --version | head -1
```

✓ 预期结果：打印 `pandoc 3.9`。不装也不影响 Markdown 日报，只是 HTML 会明确报错。

第 3 步 • 自建聊天文件，测引擎的“视力”

目的：造一份 13 条消息的“考题”（含结算、承诺、复联、纯闲聊干扰项），看规则引擎哪些答对、哪些误报。

```
# 新建 exercise/my_chat.txt，按 [YYYY-MM-DD HH:MM] 发送人：内容 格式写 13 条
# 例如："[2026-09-30 11:20] 陈总：那个方案客户通过了，尾款 12000 国庆后安排"
cd F:/reminder/wechat-intelligence-hub
python repo/projects/wechat-intelligence-hub/wechat_intelligence_hub.py scan exercise/my_chat.txt
--out exercise/out-my-scan
```

实现了什么：引擎输出 signals.csv（逐条信号：类别/阶段/优先级/证据原句）+ daily_digest.md（聚合日报）。

✅ 预期结果：13 条消息 → 7 条信号 → 6 个对象。“尾款/首款”命中结算（优先级 5），老同学合作命中“合作意向-新线索”；同时能看到误报（“明天买票”被当排期）。

⚠️ 思考点：翻 signals.csv 找一条误报和一条漏报——这就是“确定性筛子 + 人工复核”工作方式的最小演示。

第 4 步 • Markdown 日报 → 净化 HTML

目的：体验“明确报错不降级”的净化链路。

```
python repo/projects/wechat-intelligence-hub/wechat_intelligence_hub.py render-report ^
exercise/out-my-scan/daily_digest.md --out exercise/out-my-scan/daily_digest.html --title "实验
日报"
```

✅ 预期结果：打印“最终 HTML：...\daily_digest.html”，得到一个自带样式、经过 nh3 白名单消毒的独立 HTML 页面。

第 5 步 • 验证“没有授权就不读真库”的硬边界

目的：确认工具在无授权材料时明确拒绝，而不是偷偷尝试。

```
python repo/projects/wechat-intelligence-hub/wechat_intelligence_hub.py home
python repo/projects/wechat-intelligence-hub/wechat_intelligence_hub.py db-status --db
exercise/hub_demo.db
```

✅ 预期结果：两条命令分别报“找不到数据库：C:\Users\Windows\wechat-intelligence-hub\radar.db”和“找不到数据库：exercise\hub_demo.db”——**干净利落地拒绝，不降级、不猜测**。这正是 clean-room 只读承诺的工程化体现。

